Name:- Samruddhi Sangram Yadav
Roll no. B560

Title:-
Object detection using Transfer Learning of CNN architectures
a. Load in a pre-trained CNN model trained on a large dataset
b. Freeze parameters (weights) in model's lower convolutional layers
c. Add custom classifier with several layers of trainable parameters to model
d. Train classifier layers on training data available for task
e. Fine-tune hyper parameters and unfreeze more layers as needed

CODE:-

```
import os
import copy
import random
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim

from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms, models
from torchvision.models import VGG16_Weights

# ------------------------------------------------
# STEP 1: Initialize settings
# ------------------------------------------------

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("=" * 65)
print("TRANSFER LEARNING USING VGG-16")
print("=" * 65)
print("PyTorch version:", torch.__version__)
print("Device:", device)
```

```
if device.type == "cuda":
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("GPU not available. Training may be slow.")

# ----------------------------------------------
# STEP 2: Define image transformations
# Convert all images to RGB to prevent grayscale errors
# ----------------------------------------------

image_transforms = {
    "train": transforms.Compose([
        transforms.Lambda(lambda img: img.convert("RGB")),
        transforms.RandomResizedCrop(size=256, scale=(0.8, 1.0)),
        transforms.RandomRotation(degrees=15),
        transforms.ColorJitter(),
        transforms.RandomHorizontalFlip(),
        transforms.CenterCrop(size=224),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ]),

    "valid": transforms.Compose([
        transforms.Lambda(lambda img: img.convert("RGB")),
        transforms.Resize(size=256),
        transforms.CenterCrop(size=224),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ]),

    "test": transforms.Compose([
        transforms.Lambda(lambda img: img.convert("RGB")),
        transforms.Resize(size=256),
        transforms.CenterCrop(size=224),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
```

```
    ])
}

print("\nImage transformations defined successfully!")

# ------------------------------------------------
# STEP 3: Download/load Caltech-101 dataset
# ------------------------------------------------

DATA_DIR = "/content/caltech101_data"

print("\nLoading Caltech-101 dataset...")
print("The first download may take several minutes.")

train_base = datasets.Caltech101(
    root=DATA_DIR,
    target_type="category",
    transform=image_transforms["train"],
    download=True
)

valid_base = datasets.Caltech101(
    root=DATA_DIR,
    target_type="category",
    transform=image_transforms["valid"],
    download=False
)

test_base = datasets.Caltech101(
    root=DATA_DIR,
    target_type="category",
    transform=image_transforms["test"],
    download=False
)

total_size = len(train_base)
n_classes = len(train_base.categories)
class_names = train_base.categories

print("\nDataset loaded successfully!")
print("Total images:", total_size)
print("Number of classes:", n_classes)

# ------------------------------------------------
# STEP 4: Split into training, validation, testing
# 50%, 25%, 25%
```

```
# ------------------------------------------------

generator = torch.Generator().manual_seed(SEED)
indices = torch.randperm(total_size, generator=generator).tolist()

train_end = int(0.50 * total_size)
valid_end = int(0.75 * total_size)

train_indices = indices[:train_end]
valid_indices = indices[train_end:valid_end]
test_indices = indices[valid_end:]

train_data = Subset(train_base, train_indices)
valid_data = Subset(valid_base, valid_indices)
test_data = Subset(test_base, test_indices)

print("\nDataset split:")
print("Training images:", len(train_data))
print("Validation images:", len(valid_data))
print("Testing images:", len(test_data))

# ------------------------------------------------
# STEP 5: Create DataLoaders
# ------------------------------------------------

BATCH_SIZE = 16
NUM_WORKERS = 2

dataloaders = {
    "train": DataLoader(
        train_data,
        batch_size=BATCH_SIZE,
        shuffle=True,
        num_workers=NUM_WORKERS,
        pin_memory=(device.type == "cuda")
    ),

    "valid": DataLoader(
        valid_data,
        batch_size=BATCH_SIZE,
        shuffle=False,
        num_workers=NUM_WORKERS,
        pin_memory=(device.type == "cuda")
    ),

    "test": DataLoader(
```

```
        test_data,
        batch_size=BATCH_SIZE,
        shuffle=False,
        num_workers=NUM_WORKERS,
        pin_memory=(device.type == "cuda")
    )
}

# Verify image channels and tensor dimensions
sample_images, sample_labels = next(iter(dataloaders["train"]))

print("\nDataLoaders created successfully!")
print("Sample image batch shape:", sample_images.shape)
print("Expected channels:", 3, "(RGB)")

# ------------------------------------------------
# STEP 6: Load pretrained VGG-16
# ------------------------------------------------

print("\nLoading pretrained VGG-16 model...")

model = models.vgg16(weights=VGG16_Weights.DEFAULT)

# Freeze pretrained parameters
for param in model.parameters():
    param.requires_grad = False

# Replace final classifier layer
n_inputs = model.classifier[6].in_features

model.classifier[6] = nn.Sequential(
    nn.Linear(n_inputs, 256),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(256, n_classes),
    nn.LogSoftmax(dim=1)
)

model = model.to(device)

print("VGG-16 loaded successfully!")
print("Custom classifier added!")
print("\nClassifier architecture:")
print(model.classifier)

# ------------------------------------------------
```

```
# STEP 7: Define training function with early stopping
# ------------------------------------------------

criterion = nn.NLLLoss()

def train_model(model, dataloaders, criterion, optimizer,
                num_epochs=5, patience=2):

    best_weights = copy.deepcopy(model.state_dict())
    best_val_loss = float("inf")
    epochs_without_improvement = 0

    history = {
        "train_loss": [],
        "valid_loss": [],
        "train_acc": [],
        "valid_acc": []
    }

    for epoch in range(num_epochs):

        print(f"\nEpoch {epoch + 1}/{num_epochs}")

        for phase in ["train", "valid"]:

            if phase == "train":
                model.train()
            else:
                model.eval()

            running_loss = 0.0
            running_correct = 0
            total = 0

            for images, labels in dataloaders[phase]:

                images = images.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == "train"):

                    outputs = model(images)
                    loss = criterion(outputs, labels)
                    predictions = outputs.argmax(dim=1)
```

```
                if phase == "train":
                    loss.backward()
                    optimizer.step()

                batch_size = labels.size(0)

                running_loss += loss.item() * batch_size
                running_correct += (
                    predictions == labels
                ).sum().item()
                total += batch_size

            epoch_loss = running_loss / total
            epoch_acc = running_correct / total

            history[f"{phase}_loss"].append(epoch_loss)
            history[f"{phase}_acc"].append(epoch_acc)

            print(
                f"{phase.capitalize()} Loss: {epoch_loss:.4f} | "
                f"Accuracy: {epoch_acc * 100:.2f}%"
            )

            if phase == "valid":

                if epoch_loss < best_val_loss:
                    best_val_loss = epoch_loss
                    best_weights = copy.deepcopy(model.state_dict())
                    epochs_without_improvement = 0
                else:
                    epochs_without_improvement += 1

        if epochs_without_improvement >= patience:
            print("Early stopping triggered.")
            break

    model.load_state_dict(best_weights)

    return model, history

# ------------------------------------------------
# STEP 8: Train custom classifier
# ------------------------------------------------

print("\n" + "=" * 65)
```

```
print("STAGE 1: TRAINING CUSTOM CLASSIFIER")
print("=" * 65)

optimizer = optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=0.001
)

model, history = train_model(
    model,
    dataloaders,
    criterion,
    optimizer,
    num_epochs=5,
    patience=2
)

print("\nInitial classifier training completed!")

# ------------------------------------------------
# STEP 9: Fine-tune deeper convolutional layers
# ------------------------------------------------

print("\n" + "=" * 65)
print("STAGE 2: FINE-TUNING VGG-16")
print("=" * 65)

# Unfreeze the last convolutional feature layers
for param in model.features[-4:].parameters():
    param.requires_grad = True

# Keep custom classifier trainable
for param in model.classifier.parameters():
    param.requires_grad = True

optimizer = optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=0.00001
)

model, fine_history = train_model(
    model,
    dataloaders,
    criterion,
    optimizer,
    num_epochs=3,
```

```
    patience=2
)

print("\nFine-tuning completed!")

# ------------------------------------------------
# STEP 10: Evaluate using test dataset
# ------------------------------------------------

print("\n" + "=" * 65)
print("FINAL TEST EVALUATION")
print("=" * 65)

model.eval()

test_loss = 0.0
correct = 0
total = 0

with torch.no_grad():

    for images, labels in dataloaders["test"]:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)

        batch_size = labels.size(0)
        test_loss += loss.item() * batch_size

        predictions = outputs.argmax(dim=1)
        correct += (predictions == labels).sum().item()
        total += batch_size

test_loss /= total
test_accuracy = correct / total

print("Test Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")
print("Correct predictions:", correct, "/", total)

# ------------------------------------------------
# STEP 11: Plot performance curves
# ------------------------------------------------
```

```
combined_history = {}

for key in history:
    combined_history[key] = history[key] + fine_history[key]

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(combined_history["train_acc"], label="Training Accuracy")
plt.plot(combined_history["valid_acc"], label="Validation Accuracy")
plt.title("VGG-16 Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(combined_history["train_loss"], label="Training Loss")
plt.plot(combined_history["valid_loss"], label="Validation Loss")
plt.title("VGG-16 Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 12: Display sample test predictions
# ------------------------------------------------

print("\nDisplaying sample predictions...")

images, labels = next(iter(dataloaders["test"]))

model.eval()

with torch.no_grad():
    outputs = model(images.to(device))
    predictions = outputs.argmax(dim=1).cpu()

mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)
```

```
plt.figure(figsize=(12, 8))

for i in range(min(6, len(images))):

    image = images[i] * std + mean
    image = image.permute(1, 2, 0).numpy()
    image = np.clip(image, 0, 1)

    predicted_label = class_names[predictions[i].item()]
    actual_label = class_names[labels[i].item()]

    plt.subplot(2, 3, i + 1)
    plt.imshow(image)
    plt.title(
        f"Predicted: {predicted_label}\n"
        f"Actual: {actual_label}"
    )
    plt.axis("off")

plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 13: Save trained model
# ------------------------------------------------

MODEL_PATH = "/content/vgg16_caltech101.pth"

torch.save(model.state_dict(), MODEL_PATH)

print("\nModel saved successfully:", MODEL_PATH)

# ------------------------------------------------
# FINAL OUTPUT
# ------------------------------------------------

print("\n" + "=" * 65)
print("ASSIGNMENT NO. 6 COMPLETED SUCCESSFULLY")
print("=" * 65)
print("Model: VGG-16")
print("Framework: PyTorch")
print("Dataset: Caltech-101")
print("Split: 50% Training, 25% Validation, 25% Testing")
print("Transfer learning: Completed")
print("Fine-tuning: Completed")
print("Test accuracy:", round(test_accuracy * 100, 2), "%")
```

```
print("Performance graphs generated.")
print("Saved model:", MODEL_PATH)
```

OUTPUT:-

```
%runfile C:/Users/CD/.spyder-py3/test2.py –wdir
================================================================
TRANSFER LEARNING USING VGG-16
================================================================
PyTorch version: 2.11.0+cu130
Device: cuda
GPU: Tesla T4

Image transformations defined successfully!

Loading Caltech-101 dataset...
The first download may take several minutes.

Dataset loaded successfully!
Total images: 8677
Number of classes: 101

Dataset split:
Training images: 4338
Validation images: 2169
Testing images: 2170

DataLoaders created successfully!
Sample image batch shape: torch.Size([16, 3, 224, 224])
Expected channels: 3 (RGB)

Loading pretrained VGG-16 model...
VGG-16 loaded successfully!
Custom classifier added!

Classifier architecture:
Sequential(
  (0): Linear(in_features=25088, out_features=4096, bias=True)
  (1): ReLU(inplace=True)
  (2): Dropout(p=0.5, inplace=False)
  (3): Linear(in_features=4096, out_features=4096, bias=True)
  (4): ReLU(inplace=True)
  (5): Dropout(p=0.5, inplace=False)
  (6): Sequential(
    (0): Linear(in_features=4096, out_features=256, bias=True)
    (1): ReLU()
```

```
    (2): Dropout(p=0.4, inplace=False)
    (3): Linear(in_features=256, out_features=101, bias=True)
    (4): LogSoftmax(dim=1)
  )
)
```

==========
STAGE 1: TRAINING CUSTOM CLASSIFIER
==========

Epoch 1/5
Train Loss: 1.8834 | Accuracy: 57.12%
Valid Loss: 0.6986 | Accuracy: 80.41%

Epoch 2/5
Train Loss: 0.8802 | Accuracy: 75.82%
Valid Loss: 0.5315 | Accuracy: 84.28%

Epoch 3/5
Train Loss: 0.7617 | Accuracy: 78.31%
Valid Loss: 0.5118 | Accuracy: 85.38%

Epoch 4/5
Train Loss: 0.6850 | Accuracy: 81.10%
Valid Loss: 0.4680 | Accuracy: 86.08%

Epoch 5/5
Train Loss: 0.5715 | Accuracy: 83.38%
Valid Loss: 0.5739 | Accuracy: 83.73%

Initial classifier training completed!

==========
STAGE 2: FINE-TUNING VGG-16
==========

Epoch 1/3
Train Loss: 0.4792 | Accuracy: 85.02%
Valid Loss: 0.4433 | Accuracy: 86.68%

Epoch 2/3
Train Loss: 0.3821 | Accuracy: 88.29%
Valid Loss: 0.4142 | Accuracy: 87.64%

Epoch 3/3
Train Loss: 0.3308 | Accuracy: 90.20%

Valid Loss: 0.4157 | Accuracy: 87.60%

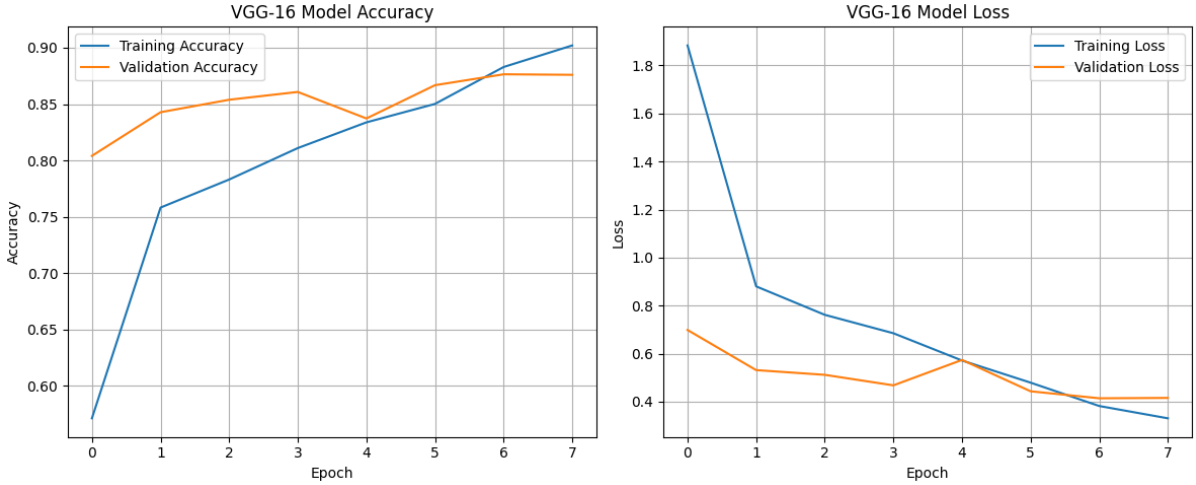
Fine-tuning completed!

==========

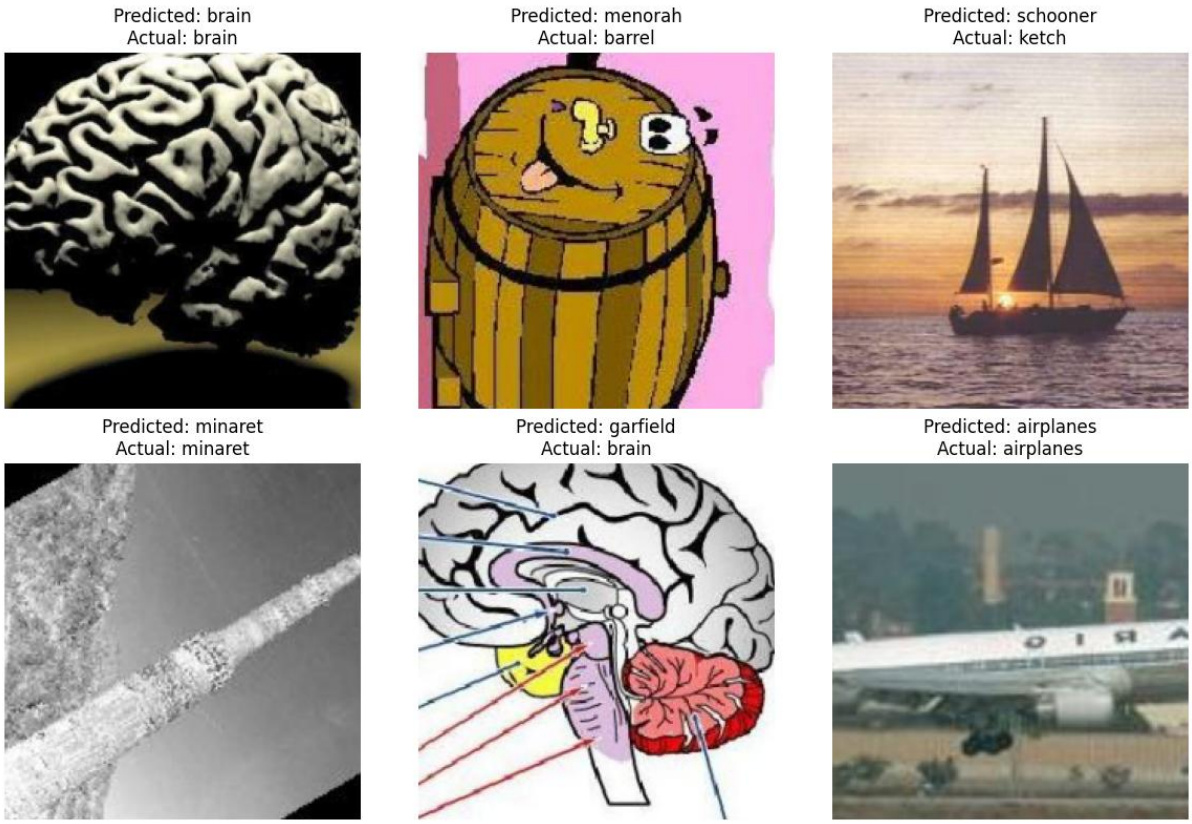
FINAL TEST EVALUATION

==========

Test Loss: 0.4616
Test Accuracy: 86.77 %
Correct predictions: 1883 / 2170



Displaying sample predictions...



Model saved successfully: /content/vgg16_caltech101.pth

==========
ASSIGNMENT NO. 6 COMPLETED SUCCESSFULLY
==========

Model: VGG-16
Framework: PyTorch
Dataset: Caltech-101
Split: 50% Training, 25% Validation, 25% Testing
Transfer learning: Completed
Fine-tuning: Completed
Test accuracy: 86.77 %
Performance graphs generated.
Saved model: /content/vgg16_caltech101.pth